LNMIIT
The LNM Institute of Information Technology

Nexura-AI

Engineering Autonomous Code with **Graph-RAG**

PRESENTED BY :

- SAKSHAM AGARWAL - 23UEC610
- SAUMIL SRIVASTAVA - 23UEC617
- YASH CHHAPARWAL - 23UEC648

SUPERVISOR :

Dr. JOYEETA SINGHA

OVERVIEW

Introduction to Nexura-AI

Redefining Autonomous Software Engineering through Structural Intelligence.

Project Purpose

An autonomous AI-powered coding assistant designed to move beyond simple autocomplete toward full-scale software evolution.

Key Innovation

Pioneering **Graph-RAG**:
Replacing unstructured text retrieval with multi-dimensional code graph analysis.

THE PARADIGM SHIFT→

CURRENT PARADIGM

Unstructured Linear Text

- ✕ Flat token sequence processing
- ✕ Probability-based word prediction
- ✕ Limited cross-file understanding

NEXURA-AI VISION

Structured Graph-RAG

- ✓ Maps how every file connects to each other
- ✓ Semantic & structural relationship mapping
- ✓ Deterministic reasoning paths

GAP ANALYSIS

The Core Problem: Limitations of Current Tools

Why existing assistants (Copilot, Cursor) struggle with complex project evolution.

Can't Connect the Dots

If the AI needs to fix something in File A, which requires checking File B, which depends on File C, it gets confused and loses track of the bigger picture.

Read code but don’t “understand” it

Current AIs read code like a normal book (just lines of text). They don't actually understand how the code pieces physically connect or work together like a blueprint.

Guess instead of checking

Code is generated without validating against existing project dependencies. Writes new code without checking if it will break other parts of your app that already exist.

Unpredictable

If you ask the AI the exact same question twice, it might give you two completely different answers. Because it's a "black box," you can't trace exactly why it made the decision it did, making it hard to trust for serious work.

Conclusion: Just guessing code based on what the AI has seen on the internet in the past isn't reliable or safe enough.

STRATEGIC ALIGNMENT

Project Objectives & Design Goals

Prioritizing control, deep understanding, and deterministic behavior.

Understands your codebase

- Learns your custom project rules and boundaries.
- Safely uses your existing files and internal tools.
- Writes new code without breaking established structures.

Connect the dots

- Traces complex problems across multiple folders at once.
- Understands indirect links between disconnected files.
- Solves issues requiring a sequence of logical steps.

Strict AI Control

- Logs every AI action step-by-step.
- Requires human review and approval for major changes.
- Prevents the AI from guessing or making rogue edits.

Foundational Research: RAG, Agents, and Verification

Key academic pillars driving the architecture of Nexura-AI.

RETRIEVAL

RAG vs. GraphRAG: A Systematic Evaluation [1]

CORE IDEA

Compares traditional vector search with relationship-based retrieval.

KEY INSIGHT

Traditional RAG misses connections; GraphRAG enables multi-step reasoning across data relationships.

🚀 **FINAL INSIGHT**

Best systems use RAG for finding data and GraphRAG for connecting and reasoning.

AGENTS

A Survey on Code Generation with LLM Agents [2]

CORE IDEA

Shift from simple code generation to autonomous agents
(Plan → Read → Write → Debug).

KEY INSIGHT

Future development relies on agents acting as software engineers with repository-level understanding.

🚀 **FINAL INSIGHT**

Future of development = AI agents acting like software engineers, not just tools.

VERIFICATION

Chain-of-Verification (CoVe) Reduces Hallucination [3]

CORE IDEA

A step-by-step self-correction loop
(Generate → Verify → Revise).

KEY INSIGHT

Reliable AI requires structured verification to detect inconsistencies and improve factual correctness.

🚀 **FINAL INSIGHT**

Reliable AI = generation + verification, not just generation.

Nexura-AI integrates these findings by combining **GraphRAG** for deep codebase context, **Autonomous Agent** workflows for engineering tasks, and **CoVe** for rigorous output validation.

DEEP DIVE: PAPER 1 Retrieval Strategy

RAG vs. GraphRAG Evaluation

THE PROBLEM

RAG treats knowledge as **flat text chunks** :

- Ignores relationships between entities
- Struggles with multi-hop reasoning
- Leads to fragmented context

ARCHITECTURE COMPARISON

RAG	Vector Search → **Top-K Chunks**
GRAPHRAG	Graph Traversal → **Subgraph**

TECHNICAL INSIGHTS

Retrieval Granularity

Retrieves **entities + edges** instead of just text fragments, enabling contextual continuity and reduced ambiguity.

Structured Context

Builds a **structured subgraph** context, explicitly traversing edges for multi-hop reasoning.

Performance Results

GraphRAG significantly outperforms RAG in **multi-hop QA** and **relational queries**, while RAG remains faster for simple factual retrieval.

NEXURA-AI RELEVANCE

Justifies treating codebases as **graphs of functions/imports**. Our system is essentially GraphRAG applied to code for deep dependency retrieval.

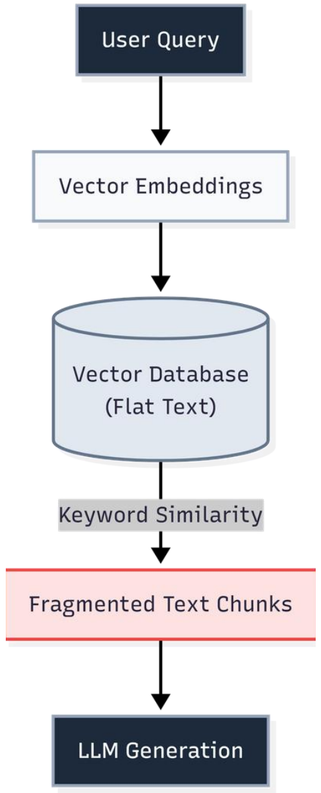
TECHNICAL DEEP DIVE

Vector RAG vs Graph RAG

From semantic similarity to structural dependency awareness.

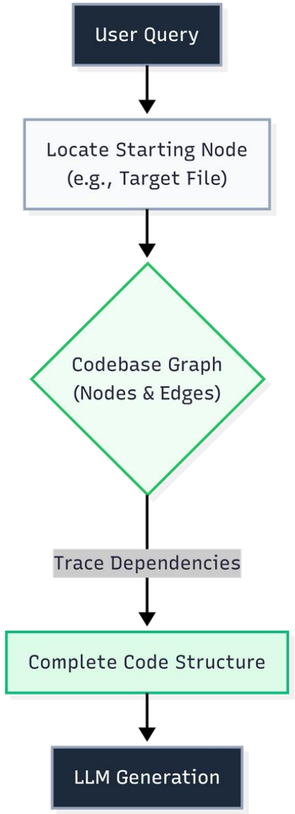
STANDARD APPROACH

Vector-Based Retrieval



NEXURA-AI INNOVATION

Graph-Based Retrieval



DEEP DIVE: PAPER 2 Agentic Frameworks

The Evolution of Autonomous Coding Agents

CORE AGENT ARCHITECTURE

1. Planner
Breaks complex tasks into manageable subtasks.

2. Retriever
Fetches relevant code, files, and documentation.

3. Executor
Writes, edits, and implements the code solution.

4. Evaluator
Tests correctness and validates the output.

THE EXECUTION LOOP

UNDERSTAND › PLAN › RETRIEVE › GENERATE › EXECUTE › EVALUATE

KEY CAPABILITIES

Repo-Level Understanding
Operates on multiple files and complex dependencies across the entire codebase.

Stateful Memory
Maintains short-term task context and long-term project knowledge.

Heavy Tool Reliance
Direct interaction with file systems, search tools, and execution environments.

NEXURA-AI RELEVANCE

Aligns with our **agentic layer** where the LLM makes decisions and acts as a digital engineer rather than just predicting tokens.

DEEP DIVE: PAPER 3 Reliability & Trust

Chain-of-Verification (CoVe)

THE COVE FRAMEWORK

1. **Draft Answer**
   Generate the initial response or code snippet based on the prompt.
2. **Verify Questions**
   The AI breaks the code down and asks itself questions to test if the logic is actually sound.
3. **Independent Answering**
   The AI answers its own test questions purely based on facts, ignoring its rough draft so it doesn't just guess.
4. **Revision**
   The AI uses the exact facts it just found to fix mistakes and writes a final, safe version.

TECHNICAL INSIGHTS

STEP-BY-STEP BREAKDOWN

Breaking big problems into tiny, testable pieces guarantees total accuracy.

OBJECTIVE FACT-CHECKING

By answering test questions without looking at the draft, the AI stops itself from making blind guesses.

RESULTS & IMPACT

- Significant reduction in factual errors
- Effective across QA and reasoning tasks
- Structured self-critique mechanism

NEXURA-AI RELEVANCE

We use this exact proofreading logic so the AI safely double-checks your project's rules before saving any new code.

MODULAR ARCHITECTURE

The Four Layer Architecture Pipeline

A structured approach to transforming source code into deterministic AI context.

LAYER 01

Graph Construction

- Reads all your project files and maps out how they connect.
- Documents exactly which files and functions rely on each other.
- Creates a master digital blueprint of your software.

LAYER 02

Graph Retrieval Engine

- Navigates blueprint to connect dots across multiple folders.
- Isolates only the specific group of files needed for the task.
- Gathers the most important code snippets to send to the AI.

LAYER 03

Reasoning & Generation

- Hands the isolated code map over to the AI.
- The AI thinks through the complex problem step-by-step.
- The AI writes new code that perfectly respects your existing rules.

LAYER 04

Verify & User Interaction

- Checks that the new code doesn't break the original blueprint.
- Uses the validators to block hallucinated or dangerous edits.
- Completely pauses the system to let a human click "Approve."

RAW SOURCE CODE > STRUCTURAL GRAPH > AGENTIC REASONING > VERIFIED EVOLUTION

END-TO-END DETERMINISTIC PIPELINE

DEVELOPMENT PHASE: V1

The Master Checklist

Enforcing determinism through the **plan.md** source of truth.

STEP 1 Query › STEP 2 Planner › STEP 3 Execute › STEP 4 Verify

The Source of Truth

Control comes from the **Master Plan**. We force the agent to document every intended action before execution begins.

- **Zero Drift:** Prevents the agent from "wandering" or losing context during complex tasks.
- **HITL Approval:** Users must review and approve the plan before code mutation.
- **Audit Trail:** Provides a clear, human-readable log of reasoning and intent.

plan.json

```
{
  "project_id": "p1",
  "goal": "Build app",
  "steps": [
    {
      "step_id": 1,
      "title": "Start",
      "description": "Initialize project",
      "dependencies": []
    }
  ]
}
```

The checklist acts as the project's safety rails, ensuring the AI performs exactly the tasks you approved and nothing more.

DEVELOPMENT PHASE: V1

Implementation: Version 1 Foundation

Building the deterministic orchestrator and safety guardrails.

RELEASE_V1.0

The Orchestration Mission

Deterministic Execution
Establishing rigid state machine logic to replace unpredictable "black-box" LLM loops.

End-to-End Observability
Full tracing of every agent node transition, tool call, and reasoning step.

Safety Guardrails
Rigid JSON schemas and node caps to prevent hallucinations and infinite tool loops.

*"V1 is about establishing **HOW** the AI operates before adding the **WHAT** of deep code intelligence."*

FOUNDATIONAL TECHNICAL STACK

LangGraph
Orchestrates the deterministic state machine nodes.

LangSmith
Provides real-time tracing and performance metrics.

FastAPI
High-speed async backend with SSE streaming.

Tauri / React
Secure local desktop frontend with FS sandbox.

THE NEXURA ANALOGY

V1 constructs the **Engine & Chassis**. Graph-RAG is the **Advanced Navigation System** added in later versions.

SYSTEM OBSERVABILITY

Observability and Tracing with LangSmith

Gaining full visibility into the agent's reasoning chains and execution lifecycle.

Full-Chain Tracing

- Step-by-step visibility into every LangGraph node execution.
- Detailed input/output logging for every LLM call and tool usage.

Metric Analytics

- Real-time tracking of **token consumption** and API costs.
- Latency analysis per node to identify performance bottlenecks.

Loop Resolution

- Detect and debug **Agentic Loops** where the LLM repeats failures.
- Dataset creation from successful traces for future fine-tuning.

TRACE HISTORY VISUALIZATION (CONCEPTUAL)

Success | 2.4s Total

Nexura_Workflow — root_id: 8f2a...

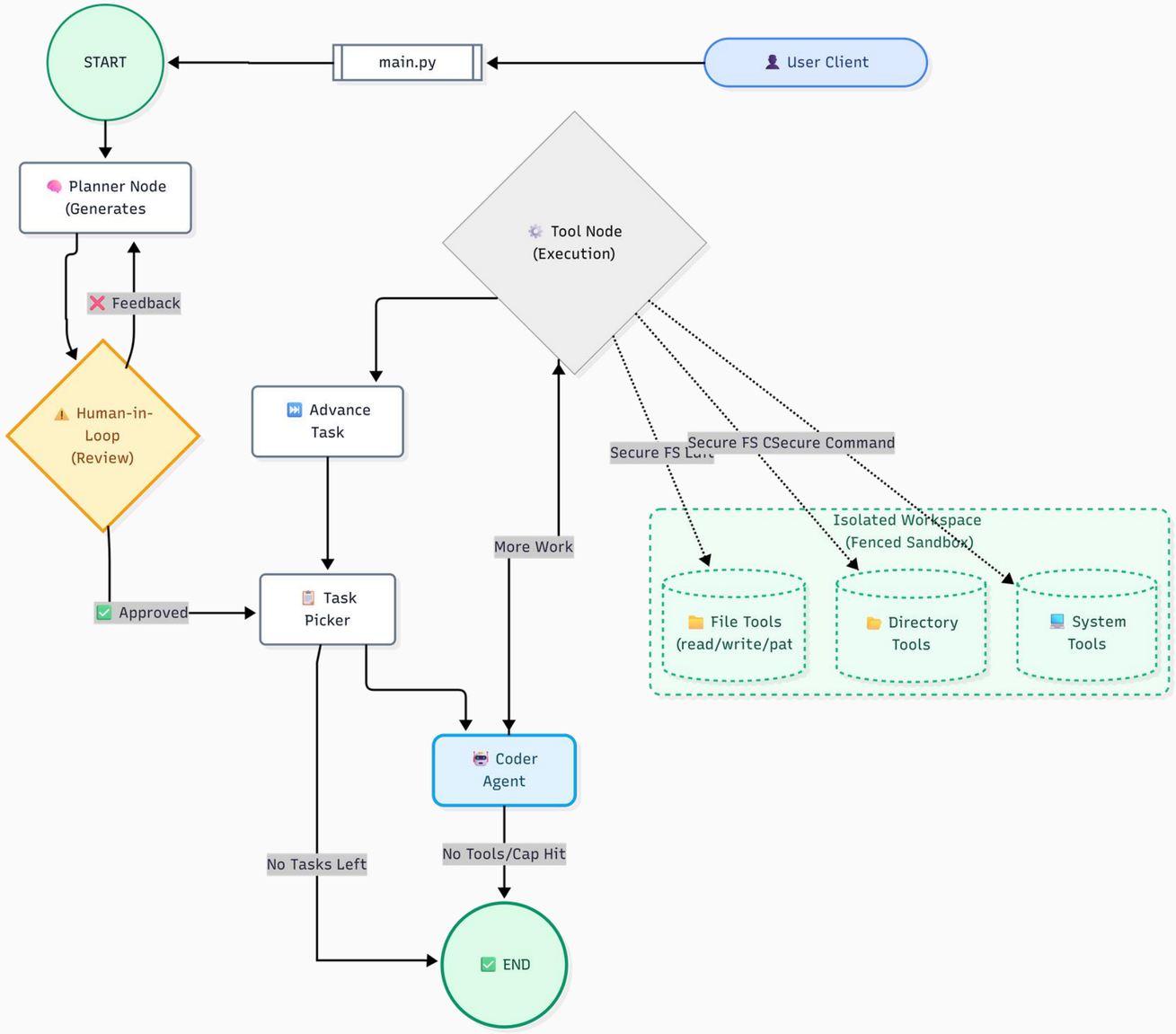
- planner_node — 450ms | 1.2k tokens
- coder_agent — 1.2s | 3.5k tokens
  - *call: write_file("app.py")* — OK
- tools — 300ms | 400 tokens

IMPLEMENTATION DETAIL | V1 FOUNDATION

Orchestration via LangGraph State Machines

Nodes & Tools

- **planner_node**
  The Architect. It reads your prompt and generates a strict, step-by-step to-do list (the plan_dict).
- **task_picker_node**
  The Manager. It looks at the approved plan, finds the next [PENDING] task, and hands exactly one task to the coder.
- **coder_agent_node**
  The Worker. The actual AI model. It reads the single task, figures out what code needs to be written, and requests permission to use tools.
- **tools**
  It physically executes the commands requested by the Coder Agent (like saving a file to your hard drive).
- **advanced_task_node**
  Once the Coder finishes a task, this node crosses it off the to-do list (marks it [COMPLETED]) and sends the system back to the Task Picker for the next job.



IMPLEMENTATION PHASE

Frontend and API Implementation

Building a responsive, real-time interface for autonomous agent orchestration.

ARCHITECTURE STACK

Tauri Desktop Application Wrapper (Rust-based)

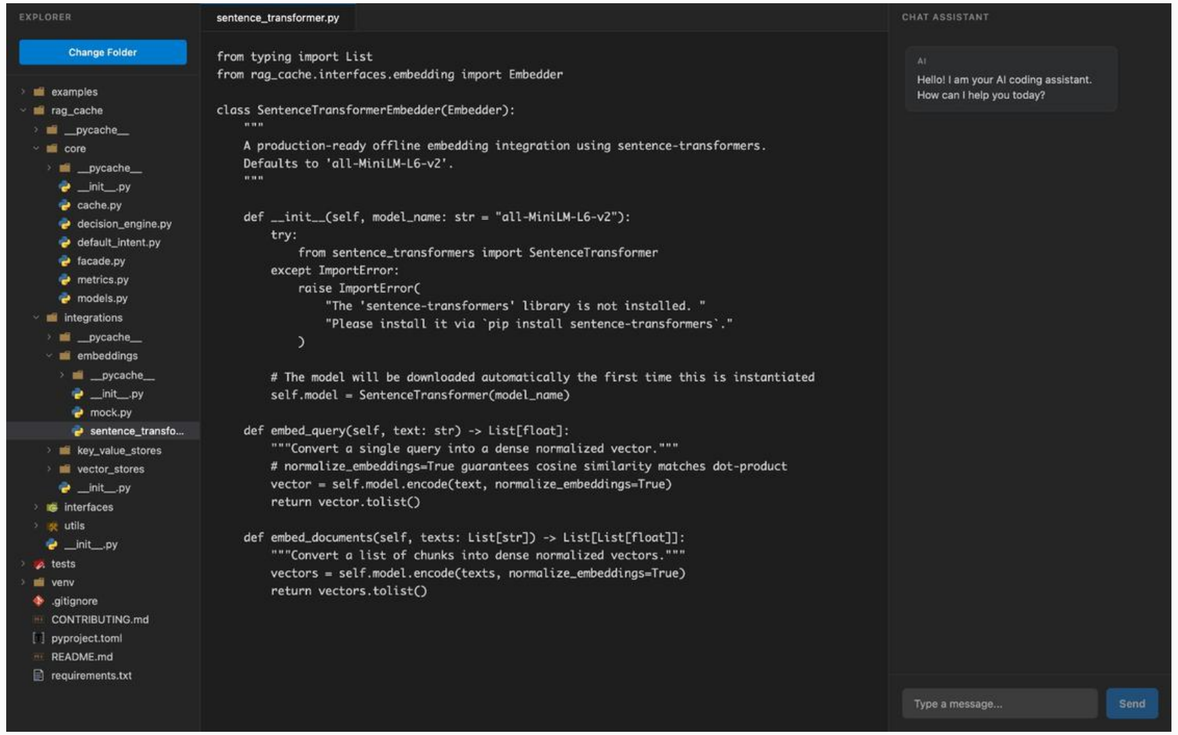
React Dynamic UI & State Management

FastAPI High-performance Backend Orchestrator

Key UI Features

- **Interactive Terminal:** Visualizes agent "thoughts" and tool outputs.
- **File Diff Viewer:** Side-by-side comparison of AI-generated changes.
- **HITL Controls:** Manual approval buttons for plan execution.

USER-INTERFACE (UI)



Demonstration: Agent Workflow in Action

Live visualization of autonomous planning, multi-file execution, and automated verification.

1

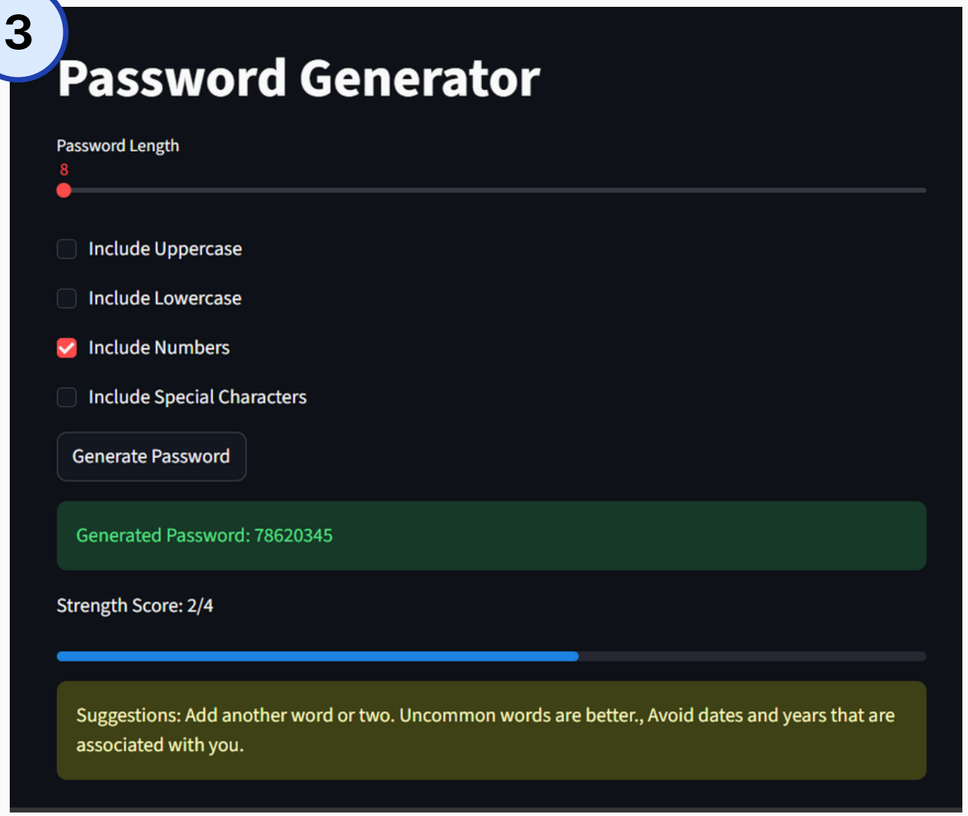
```
(venv) PS C:\Users\saumi\Nexura-AI\workspace\proj_password> python main.py --project "proj_password" --prompt "Create a Streamlit app that is a strong password
generator. It should have sliders for length, checkboxes for numbers and symbols, and a 'Generate' button. Below that, write a python function that calculates '
Password Strength' (weak, medium, strong) based on length and characters, and display that result using Streamlit progress bars."
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF { } Python 3.13.5 venv (3.13.5) Prettier

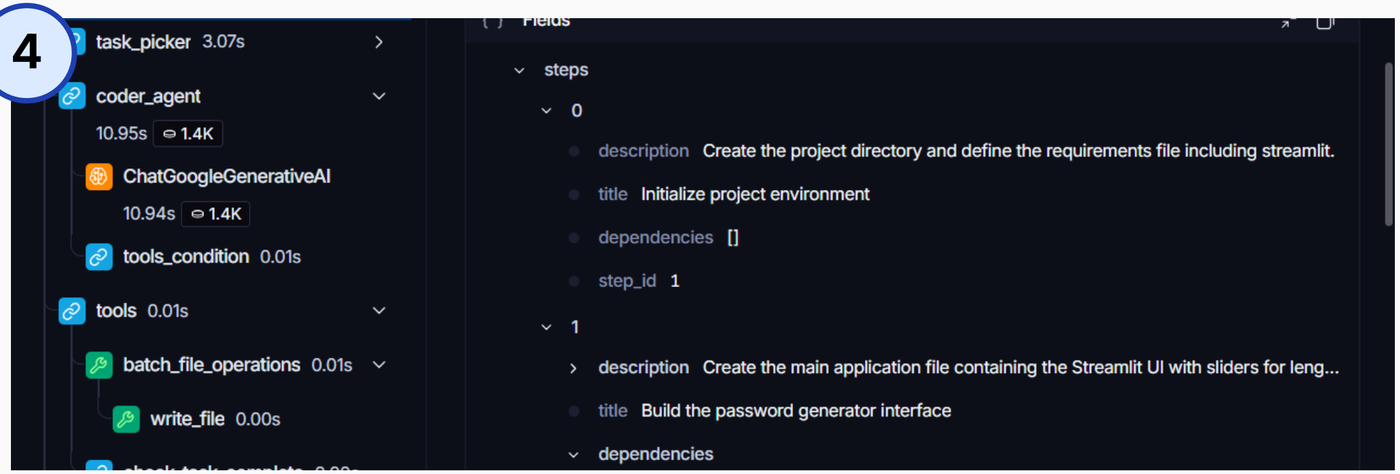
2

```
==================================================
GENERATED PLAN
==================================================
Project: streamlit-password-generator
Goal: Build a Streamlit-based password generator with strength analysis and visual feedback.

Steps (4):
  1. Initialize project environment
     Create the project directory and define the requirements file including streamlit.
  2. Build the password generator interface (depends on: [1])
     Create the main application file containing the Streamlit UI with sliders for length, checkboxes for character sets, and a generation button.
  3. Define password strength logic (depends on: [2])
     Write a Python function that evaluates password strength based on length and character variety, returning a score.
  4. Integrate strength visualization (depends on: [3])
     Update the application to display the generated password and render the strength score using Streamlit progress bars.
==================================================

>> Review the plan above. Type 'yes' to approve, or provide feedback to re-plan:
>> yes
```

3



4



PHASE 02

Future Roadmap: Version 2

Transitioning from structural parsing to semantic comprehension.

Local Embeddings

The AI will act like a smart search engine for your project. You can ask it to "find the user login logic," and it will find it instantly, even if you never said the exact word "login."

Vectorization Semantic Indexing

Long-Term Memory

The AI will finally remember you. It will learn your coding style, remember past mistakes to avoid repeating them, and track your preferences across different coding sessions.

Pattern Storage User Prefs

Semantic Search

The AI will automatically pull in related files across different folders, giving it a much deeper understanding of how your project fits together.

Vector Store Context Ranking

ARCHITECTURAL EVOLUTION

Expanding the Contextual Frontier

1 **Controlled Pipeline**
V1 Foundation

2 **Semantic Embedding**
Neural Representation

3 **Structural Graph**
Persistent Learning

IMPACT
90% Reduction in Context Blindness

© 2026 NEXURA ENGINEERING

ROADMAP PHASE 3 FUTURE VISION

Version 3: Autonomous Graph-RAG

Transitioning from assisted generation to fully autonomous CI/CD integration.

V3 Strategic Focus

- **Full Graph-RAG Deployment:**
  Leverage the complete code graph for multi-hop reasoning and dependency-aware logic.
- **GitHub MCP Integration:**
  Model Context Protocol for direct interaction with repositories and CI/CD pipelines.
- **Learning Memory:**
  Continuous refinement of graph retrieval based on interaction success rates.

Key Deliverables

- Autonomous Branch Management
- Self-Healing Test Suites
- Automated PR & Merging Logic
- Security Vulnerability Auto-Fix

DEVELOPMENT EVOLUTION

V1: Foundation	V2: Context	V3: Automation
Orchestration & Sandbox	Embeddings & Memory	Graph-RAG & GitHub MCP

ENGINEERING REALITIES

Current Limitations & Trade-offs

Acknowledging the inherent challenges of moving from vector-based retrieval to structural graph-based reasoning.

Construction Overhead

Forcing the AI to plan and double-check its actions before typing takes extra time upfront.

- Takes longer to start
- Uses more computing power
- Constantly updates its checklist

Reasoning Latency

tl checks its work against the rules at every single step & responds much slower.

- Slower immediate responses
- Constant double-checking
- Requires users to be patient

Memory Footprint

Keeping logs (langsmith) of every single AI action and system state requires extra processing power.

- High RAM for local execution
- Harder to scale infinitely
- Constant tracing adds backend work

The Fundamental Trade-off

Trading raw speed for architectural precision and deterministic control.

Accuracy PRIORITIZED VS **Velocity** SACRIFICED

Conclusion & References

Final summary and foundational research citations

NEXURA-AI // PROJECT_FINAL

PROJECT SUMMARY

The Problem

Current assistants lack deep structural understanding and multi-hop reasoning, leading to unpredictable outputs.

Our Philosophy

Nexura-AI champions **Control > Intelligence**, ensuring deterministic and verifiable code generation.

V1 Foundation

Established a robust orchestrator using LangGraph and LangSmith for secure, observable, and auditable workflows.

Future Vision

Roadmap to V2/V3 introduces full Graph-RAG and seamless CI/CD integration via GitHub MCP.

FOUNDATIONAL RESEARCH

01 *[1] Y. Liu, X. Zhang, and J. Wang, **"RAG vs. GraphRAG: A Systematic Evaluation and Key Insights,"** arXiv preprint arXiv:2405.16506, 2024.*

02 *[2] Z. Chen, L. Huang, and Y. Liu, **"A Survey on Code Generation with LLM-based Agents,"** arXiv preprint arXiv:2508.00083, 2025.*

03 *[3] S. Dhuliawala, M. Chen, J. White, and S. Singh, **"Chain-of-Verification Reduces Hallucination in Large Language Models,"** arXiv preprint arXiv:2309.11495, 2023.*

04 *[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, **"Attention Is All You Need,"** in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.*

05 *[5] Y. Zhang, X. Chen, and L. Wang, **"GRAG: Graph Retrieval-Augmented Generation,"** arXiv preprint arXiv:2405.16506, 2024.*

THANK YOU